

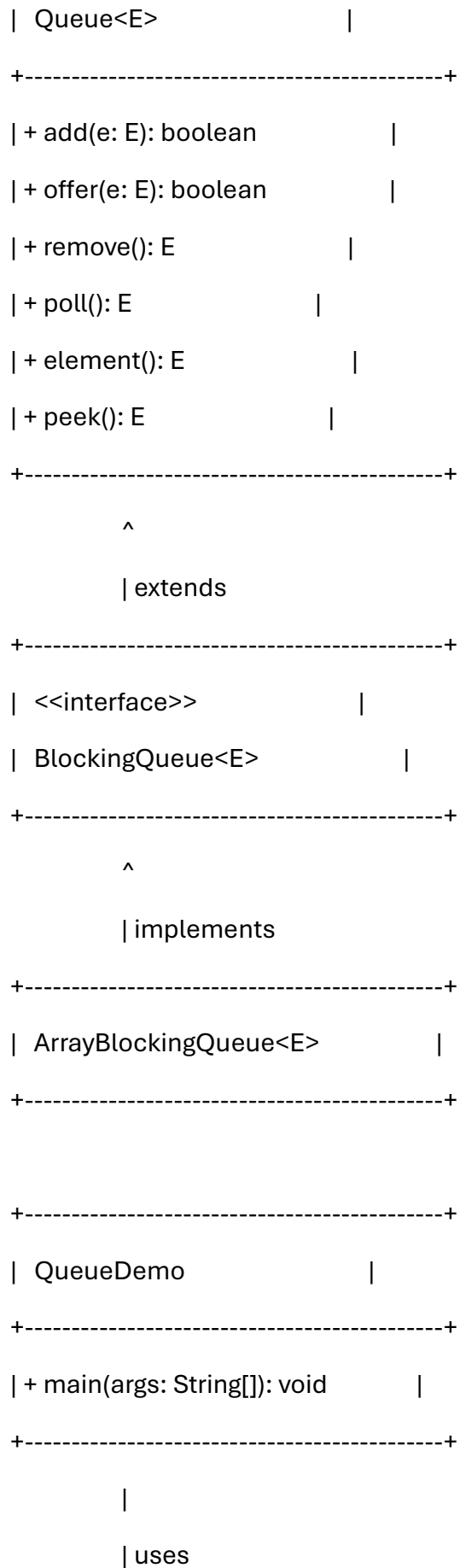
Aim:

To write a Java program that demonstrates the methods of the Queue interface, namely add(), offer(), remove(), poll(), element() and peek(), using an ArrayBlockingQueue of fixed capacity to show how the methods that throw exceptions differ from the methods that return a special value (false or null) when an operation fails.

Algorithm:

1. Start.
2. Create a Queue of String using ArrayBlockingQueue with capacity 3.
3. Call add() with "A" and "B", and offer() with "C". Each returns true.
4. Display the queue. The queue is now full.
5. Call offer("D"). It returns false because the queue is full.
6. Call add("D") inside a try-catch block. It throws IllegalStateException.
7. Call element() and peek() to view the head of the queue. Neither removes it.
8. Call remove() and poll() to retrieve and remove the head twice, then display the queue.
9. Call poll() once more to empty the queue.
10. Call poll() and peek() on the empty queue. Both return null.
11. Call remove() and element() on the empty queue inside try-catch blocks. Both throw NoSuchElementException.
12. Stop.

UML Diagram:



v

Queue<E>

Code:

```
package Observations;

import java.util.concurrent.ArrayBlockingQueue;
import java.util.NoSuchElementException;
import java.util.Queue;

public class QueueInterface {

    public static void main(String[] args) {

        Queue<String> queue = new ArrayBlockingQueue<>(3);

        System.out.println("add(\"A\"): " + queue.add("A"));
        System.out.println("add(\"B\"): " + queue.add("B"));
        System.out.println("offer(\"C\"): " + queue.offer("C"));
        System.out.println("Queue: " + queue);

        System.out.println("offer(\"D\") on full queue: " + queue.offer("D"));

        try {

            queue.add("D");

        } catch (IllegalStateException e) {

            System.out.println("add(\"D\") on full queue: IllegalStateException - "+
e.getMessage());

        }

        System.out.println("element(): " + queue.element());

        System.out.println("peek(): " + queue.peek());
```

```
System.out.println("Queue after element() and peek(): " + queue);
```

```
System.out.println("remove(): " + queue.remove());
```

```
System.out.println("poll(): " + queue.poll());
```

```
System.out.println("Queue after remove() and poll(): " + queue);
```

```
System.out.println("poll(): " + queue.poll());
```

```
System.out.println("Queue: " + queue);
```

```
System.out.println("poll() on empty queue: " + queue.poll());
```

```
System.out.println("peek() on empty queue: " + queue.peek());
```

```
try{
```

```
    queue.remove();
```

```
} catch (NoSuchElementException e) {
```

```
    System.out.println("remove() on empty queue: NoSuchElementException");
```

```
}
```

```
try{
```

```
    queue.element();
```

```
} catch (NoSuchElementException e) {
```

```
    System.out.println("element() on empty queue: NoSuchElementException");
```

```
}
```

```
}
```

```
}
```

Output:

```
es' '-cp' 'C:\Users\Sahithi pyla\AppData\Roaming\Code\User\workspaceStorage
bservations.QueueInterface'
add("A"): true
add("B"): true
offer("C"): true
Queue: [A, B, C]
offer("D") on full queue: false
add("D") on full queue: IllegalStateException - Queue full
element(): A
peek(): A
Queue after element() and peek(): [A, B, C]
remove(): A
poll(): B
Queue after remove() and poll(): [C]
poll(): C
Queue: []
poll() on empty queue: null
peek() on empty queue: null
remove() on empty queue: NoSuchElementException
element() on empty queue: NoSuchElementException
PS C:\Users\Sahithi pyla\Desktop\Sahithi_java>
```

Result:

The program was executed successfully. On a full queue, offer() returned false while add() threw an exception. On an empty queue, poll() and peek() returned null while remove() and element() threw NoSuchElementException. element() and peek() did not remove the head, whereas remove() and poll() did.